

CS4395 Assignment 2

<https://github.com/nicodlc/CS4395-Assignment2>

Nicolás De la Cruz Velasquez

Net ID: nad220002

1. Introduction and Data (5pts):

- 1.1. This project introduces us with a dataset of different text samples with a relevant score in the form of `{text: string, score: number}`. Each text sample is associated with a relevant score, and both serve as inputs for two different NN architectures; a simple Feed Forward NN and a Recurrent NN, for a sentiment analysis task. All data is vectorized [or generated as embeddings] before going into the neural network.

Some of the details in the preprocessing stage of the project include two separate methods to obtain embeddings. The FFNN uses a fixed length vector which is performed by a bag-of-words vectorization, while the RNN uses another embedding model saved as a pickle file in the directory we're working under.

1.2. Information on the data used:

Data set type	Sample count
Training	5000
Testing	800
Validation	800

1.3. Main, or default, experimental results:

- 1.3.1. **Feed Forward Neural Network:** ran with the command `python ffnn.py --hidden_dim 10 --epochs 10 --train_data ./training.json --val_data ./validation.json`.

The best achieved validation accuracy score throughout the 10 epochs is around 0.6025.

1.3.2. **Recurrent Neural Network:** ran with the command: `python rnn.py --hidden_dim 32 --epochs 10 --train_data training.json --val_data validation.json`.

The best achieved validation accuracy score throughout the 10 epochs is around 0.46625, stopped at epoch=2 to avoid overfitting.

2. **Implementations (45pt):** Tiny disclaimer: most of the math involved is extracted directly from the PyTorch documentation and I'm just grabbing the definitions to sequentially fill in the gaps over the code. Apologies if my LaTeX is messy.

2.1. **FFNN (20pt):** This is a screenshot constituting the implementation of the missing forward pass function on the *Feed Forward Neural Network*.

```
def forward(self, input_vector):
    # [to fill] obtain first hidden layer representation
    hidden = self.W1(input_vector)
    act_h = self.activation(hidden)
    # [to fill] obtain output layer representation
    outp = self.W2(act_h)
    # [to fill] obtain probability dist.
    predicted_vector = self.softmax(outp)

    return predicted_vector
```

2.1.1. Parameters:

- The input data `input_vector` is a d -dimensional vector, $x \in R^d$ where d is the vocabulary size.
- The hidden layer `hidden` $h \in R^{|h|}$ where $|h|$ is its dimension.
- The output layer `outp` $z \in R^{|\gamma|}$ where $|\gamma|$ is its dimension.

2.1.2. Operations:

- The first linear activation is given inside `hidden` with it being $hidden = W_1^T x + b_1$ inside `W1` calling `nn.Linear()`
- The activation function receiving `hidden` is dictated by `act_h` containing $a^{(1)} = \max(0, hidden^{(1)})$ or more commonly known as `nn.ReLU()`.

- The output layer `outp` receives the contents of the activation function from the hidden layer, as described in

$$outp = W_2^T a^{(1)} + b_2$$

- The final stage of the operations includes the application of Log Softmax using the output layer as argument:

$$predictedVector = \log\left(\frac{\exp(outp_i)}{\sum_{j=1}^5 \exp(outp_j)}\right), \text{ to compute the finalized}$$

`predicted_vector` variable that is returned by the function `forward()`.

- 2.2. **RNN (25pt):** Another screenshot depicting the implementation of the missing segment on the RNN class, the forward function:

```
def forward(self, inputs):
    # [to fill] obtain hidden layer representation (https
    _, hidden = self.rnn(inputs)
    # [to fill] obtain output layer representations
    hidden_state = hidden[0]
    # [to fill] sum over output
    linear_output = self.W(hidden_state)
    # [to fill] obtain probability dist.
    predicted_vector = self.softmax(linear_output)

    return predicted_vector
```

- 2.2.1. Parameters:

- The `inputs` variable carries over a sequence of vectors $x_1, x_2 \dots x_k, x_i \in R^e$ where e is the embedding size.
- Representations after RNN, a new sequence of updated representation of the input vectors, $h_1, h_2, \dots h_k, h_i \in R^{|h|}$, where $|h|$ is the hidden layer dimension.
- Output layer is carried on by the `linear_output` variable, where it is defined as $\sum_{i=1}^k z_i, z_i \in R^{|\gamma|}$, where $|\gamma|$ is its dimension.

- 2.2.2. Operations:

- All inputs in `input` are passed into the RNN, which internally modifies them using the following function [for each layer]:

$$h_t = \tanh(x_t W_{ih}^T + b_{ih} + h_{t-1} W_{hh}^T + b_{hh})$$
The final hidden state is represented as $h_{final} = h_T$ where I'm only extracting the first row of the hidden state tensor inside the RNN, corresponding to the code line `hidden_state = hidden[0]`. This is just to not overdo processing by considering each layer outputs to then be mapped into some h_i , `hidden[0]` takes the form of the compressed summary of the sequence.
- Using the `hidden_state` variable, I apply a linear transformation that receives the variable, and by default maps it into a vector $\in R^5$
`self.W = nn.Linear(h, 5)`. Mathematically, it would look something like $linearOutput = Wh_{final} + b$ where $W \in R^{5 \times h}$ and $b \in R^5$. This is all encapsulated in the line
`linear_output) = W(hidden_State)`.

2.3. **Overall conclusion on 2.1 and 2.2:** So, what I get from both implementations is that the Feed Forward NN does not apply transformations iteratively on the hidden features compared to the RNN. All transformations in a FFNN are applied in a single forward pass from the input to the output, while the RNN uses the transformed hidden state of the previous step! That makes sense since the RNN takes more time to calculate and make predictions.

Moreover, one can clearly notice a tradeoff whenever running the code. Preprocessing with two different approaches as to how we obtain embeddings can definitively impact the performance when training the models.

3. Experiments and Results!

3.1. Evaluations (15pt)

- 3.1.1. **FFNN Evaluations:** The current default parameters of the model specify a 10-dimensional hidden layer. Additionally, the default number of epochs specified is just a single one. This is enough to actually show how the forward pass takes place, but to no avail if we want the model to 'learn'. It is to my understanding that the increase in dimensionality of the hidden layer might yield better results on training, or better capture the patterns seen on the text. I'll try to gradually increase the number of dimensions for

the same number of epochs.

- 3.1.2. **RNN Evaluations:** The default parameters passed onto the model specify a 32-dimensional hidden layer. Additionally, the default number of epochs is 10, assuming that it is due to how long it takes to compute through everything. I could also infer that the dimension of the hidden layer is the one that determines how many patterns are captured through each iteration. Since it moves sequentially with each state from the last layer, it'd make sense to try and fit as many of the mentioned patterns as possible.

3.2. Results (30pt):

3.2.1. FFNN Results:

Epochs	Hidden Dimensions	Best training accuracy	Best validation accuracy
15	10	0.78663	0.6025
15	16	0.8060	0.6237
15	22	0.8381	0.6310
15	24	0.8112	0.6175
15	28	0.83463	0.62
15	32	0.8228	0.63125

So far, the gradual increase on the hyperparameter count, specifically for the hidden dimensions, seems to aid in the generalization as the model takes in more parameters to extract and learn from the data. As reflected in the table, the training accuracy has had around a 5% increase as the hidden dimension count has tripled.

On the other hand, the validation accuracy has only had a 3% increase with the new parameters being added. It is relieving to see that we get the highest validation accuracy as the last, biggest count of hidden features. We might be facing some overfitting in contrast to the training's greater results. Unseen data seems to still be an issue to consider.

3.2.2. RNN Results: Disclaimer: code kept stopping at epoch = 2 so I commented it out to let it run for a bit.

Epochs	Hidden Dimensions	Best training accuracy	Best validation accuracy
10	10	0.76663	0.5925
10	16	0.8060	0.6237
10	22	0.8381	0.6310
10	24	0.8294	0.62140
10	28	0.8129	0.6182
10	32	0.8228	0.63125

For the Recurrent NN, the training results yield just a slighter improvement in training results, going on a 7% rise as the parameters are almost tripled. Note that the best/peak performance is around 22 hidden dimensions. This probably indicates that providing more capacity on each dimension state can better help represent dependencies as the model processes them sequentially.

On the other hand, the validation results are surprisingly similar to the ones shown by the FFNN, hinting towards a 4% increase as the number of hidden features also increases. Considering that it took a while to render all results, I would've been surprised if it didn't increase in accuracy at all. Since we're taking a past state and carrying it forward, I'd estimate it would learn better at the expense of computational power.

4. Conclusion

The overall difficulty of the assignment was decent. The implementations made for both the FFNN and RNN architectures do resemble a challenge if you don't refer to PyTorch's documentation. I found it to be extremely useful when trying to make sense of the math and the inner tweaks one can make. The class materials also have yielded a pretty good summary of some of the best practices for this project and its goal.

Time-wise, it honestly has been a challenge. The FFNN did not take much time to preprocess and fit the data, since it is doing computations in parallel for its hidden layers. However, the training done on the RNN is definitely correlated to the complexity of the

assignment. Not only because it can be slow if running on a low-end computer, but also trying to monitor and record each training measure compared to its previous result. Each hidden state took about 1 or 1.5 minutes to process.

Both models obtained similar results, at the tradeoff of time consumption and resource allocation in the case of different vectorization methods. The increase of hidden features in the RNN meant that the dimension of the input vectors would also have to match. This increased the preprocessing time before the actual execution, but delayed the execution time as well to process each layer as correctly as possible.